OPERATING SYSTEMS (UE25CS242A)

Unit 1 – Introduction and Process Management

Question and Answers

Lecture 1 - Introduction

Q1. You are designing an Operating System for a supercomputer used exclusively for weather forecasting (a scientific mainframe) and another for a home entertainment tablet. Contrast the "Operating System Goals" for these two systems. How does the definition of "efficiency" differ between them?

Answer: **Mainframe (Weather Supercomputer):** The primary goal is Resource Utilization. Since the hardware is expensive and powerful, the OS must ensure the CPU, memory, and I/O channels are busy constantly to maximize throughput. "Efficiency" here means executing the maximum number of calculations per second.

Tablet (Home Entertainment): The primary goal is User Convenience and Ease of Use. The OS focuses on a responsive UI and battery life rather than keeping the CPU 100% busy.

"Efficiency" here means low power consumption and smooth interaction, even if the CPU sits idle waiting for user input.

Q2. Why is the Operating System described as "Interrupt Driven"? What would happen to the system's responsiveness if interrupts were disabled entirely?

Answer: Interrupt Driven: The OS sits idle until something happens. Events (keystrokes, disk completion, network packets, system calls) are all signaled via interrupts. The OS wakes up, handles the event, and goes back to waiting or running a user process.

Consequence of Disabling: If interrupts were disabled, the OS could not react to any external input. The keyboard would freeze, the mouse wouldn't move, and disk operations wouldn't notify the CPU upon completion. The system would effectively appear "hung" or unresponsive to the user.

Q3. "A user program cannot directly access hardware." If a text editor needs to save a file to the disk, explain the mechanism it uses to bypass this restriction securely. What role does the Dual-Mode (User/Kernel) architecture play here?

Answer: The text editor issues a System Call (e.g., write()). This is a special instruction that triggers a software interrupt (Trap).

Dual-Mode Role:

- The trap switches the CPU from User Mode to Kernel Mode.
- The OS (now in Kernel Mode) validates the request (permissions, path).
- The OS performs the hardware operation on the disk.
- The OS switches back to User Mode and returns control to the text editor.
- This ensures the user program never touches the hardware directly, preventing errors or malicious access.

Lecture 2 - Computer-System Architecture, Operating-System Structure & Operations

Q1. In a Symmetric Cluster, two nodes try to write to the same student record on the shared Storage Area Network (SAN) at the exact same instant. What mechanism prevents data corruption here? Why isn't a simple local semaphore sufficient?

Answer: Since the nodes are separate computers, they do not share local memory. A local semaphore on Node A is invisible to Node B. Thus, the system must use a Distributed Lock Manager (DLM).

Mechanism: The DLM coordinates access control across the network, ensuring that only one node holds the "write lock" for a specific shared resource on the SAN at any given time.

Q2. In a multiprogrammed system, multiple jobs reside in memory simultaneously. If the system has 8GB of RAM but the active jobs require 12GB total, what mechanism allows the system to continue operating? Briefly explain Swapping/Virtual Memory.

Answer: Constraint: Physical memory is finite. Not all jobs can fit in RAM at once.

Mechanism: The OS uses Swapping or Virtual Memory.

Operation: Processes not currently executing are swapped out to the hard disk (backing store), freeing up RAM for the active process. When the swapped-out process is needed again, it is loaded back into memory. This allows the execution of processes larger than physical memory.

Q3. Scenario: What would happen if the Mode Bit stuck at '0' due to a hardware glitch while a user program was running? Analyze the risk to system stability and security.

Answer: State: If the bit sticks at 0, the CPU remains in **Kernel Mode** permanently.

Risk: The currently running user program would have unrestricted access to the entire system.

Consequence: It could read other users' passwords from memory, wipe the hard disk, or overwrite the OS code. The system loses all protection boundaries, likely leading to a crash or security breach.

Q4. Modern CPUs often support more than just two modes. What is the purpose of the VMM (Virtual Machine Manager) Mode? How does it relate to "Guest VMs"?

Answer: In virtualization, a host OS runs multiple guest operating systems. CPUs support a dedicated mode for the **Virtual Machine Manager (Hypervisor).**

This allows the VMM to have higher privileges than the Guest OS kernels (which think they are in kernel mode but are actually being managed) without needing full physical root access for every operation, improving virtualization efficiency and security.

Lecture 3 - OS Services, Process Concept

Q1. A headless server (a computer without a monitor or keyboard) in a data center crashes due to a memory error. Since there is no screen to display an error message, which OS service is responsible for handling this, and how does the administrator eventually find out what happened?

Answer: The Error Detection service is responsible. When the error occurs, the OS generates a core dump or writes the error details to a system log file.

The administrator accesses these logs later (via network or remote shell) to diagnose the issue. Without this service, the system would simply freeze with no diagnostic trail.

Q2. Early OSes were written in Assembly. Modern OSes (like Linux/Windows) are written mostly in C/C++. Give two reasons for this shift. Is there any part of the OS still written in Assembly?

Answer: C code is **portable.** C code can be recompiled for different CPU architectures (x86, ARM). Assembly is specific to one CPU family.

It is also highly **maintainable**. High-level code is easier to read, write, and debug.

Yes, small critical sections like the **interrupt handlers** and **device drivers** are still written in Assembly for maximum speed and direct hardware control.

Q3. Two processes need to exchange data. The OS provides Shared Memory and Message Passing. Which method would you choose for exchanging small amounts of status data between two computers across a network? Justify.

Answer: Message Passing.

Justification: Shared Memory is difficult or impossible to implement across different physical computers (distributed systems). Message passing is designed for communication, handling the packet formatting and synchronization required for network data transfer.

Q4. How does the "Process Lifecycle" in a mobile OS (like Android) differ from a traditional desktop OS regarding background processes? Why might the OS terminate a background process without the user explicitly closing it?

Answer: Mobile devices have limited memory and battery compared to desktops. While desktop systems typically use swapping (virtual memory) to manage overflow, mobile OSes generally do not support swapping.

Instead, when memory runs low, the mobile OS automatically terminates background processes

(that are not visible to the user) to reclaim memory for the foreground application. The state is saved to flash memory to be restored later.

Q5. System Calls: Explain the relationship between fork() and exec(). Task: Why do most programs call exec() immediately after fork()? What happens to the memory of the child process during exec()?

Answer: fork() creates a duplicate copy of the parent process. exec() replaces the memory space of a process with a new program. Programs call them in sequence to run a different program. For example, the Shell forks a copy of itself, then the child calls exec() to become the "ls" command. During exec(), the child's text, data, and stack segments are completely destroyed and overwritten by the new program loaded from the disk.

Q6. What is "Cascading Termination"? If a user kills the main BASH shell process, what happens to the background commands running under it in an OS that enforces cascading termination?

Answer: Cascading Termination is a policy where the OS does not allow a child process to exist if its parent has terminated.

If the BASH shell (Parent) is killed, the OS detects its children. The OS automatically initiates termination for all children, grandchildren, and further descendants of that shell. All background commands running under it are forcibly closed.

Lecture 5 - Process Creation and Termination, System calls for process management

Q1. You are optimizing a system where a process needs to create a child that immediately calls exec() to run a new program. Would you use fork() or vfork()? Justify your choice based on memory overhead and performance.

Answer: vfork() is preferred in this specific scenario.

Justification:

● **Standard fork():** Copies the parent's data space, heap, and stack to the child. If the child immediately calls exec(), this copying is wasted effort because exec() replaces all of it anyway.

● **vfork() Optimization:** It creates a child without copying the address space. The child borrows the parent's memory until it calls exec(), making it significantly faster and more memory-efficient for this specific pattern.

Q2. A server runs for months. You check the process table and see hundreds of "Zombie" entries. Explain what a Zombie process is in terms of resource usage. Why is it a problem if they accumulate?

Answer: A Zombie is a process that has terminated (exit()) but whose parent has not yet called wait() to collect its status. They consume no CPU and almost no memory (text/data are freed). They still occupy a slot in the **Process Table** (to store the PID and exit status). If zombies accumulate, the system runs out of available PIDs, preventing new processes from starting.

Q3. "During I/O execution, the requesting process is typically blocked." Task: Why does the OS block the process instead of letting it spin in a loop checking for data? How does this improve "System Throughput"?

Answer: I/O devices are thousands of times slower than the CPU. Spinning (busy waiting) would waste millions of CPU cycles doing nothing. By blocking the process (moving it to the Waiting state), the OS frees the CPU.

The OS performs a Context Switch to run a different, ready process. This keeps the CPU busy doing useful work for Process B while Process A waits for the disk, increasing overall system throughput.

Lecture 7 - CPU Scheduling- FCFS Scheduling, SJF Scheduling

Q1. Analyze the "CPU-I/O Burst Cycle" and its significance in the context of multiprogramming. How does the frequency distribution of CPU bursts influence the design of scheduling algorithms?

Answer: To understand why CPU scheduling is necessary, one must first understand the lifecycle of a process. Process execution is not a continuous block of calculation; rather, it consists of an alternating cycle of **CPU execution** (CPU burst) and **I/O wait** (I/O burst).

In a system without multiprogramming, when a process encounters an I/O burst (like reading from a file), the CPU sits idle, wasting valuable computational resources. The fundamental objective of multiprogramming is to maximize CPU utilization by switching the CPU to another process effectively during these wait times.

The **frequency distribution of CPU** bursts is critical for designing efficient schedulers. Empirical measurements show that processes tend to have a vast number of short CPU bursts (I/O-bound programs) and a small number of long CPU bursts (CPU-bound programs). A scheduling algorithm must be designed to handle this curve effectively—balancing the needs of interactive processes (short bursts) against computational heavyweights (long bursts) to ensure the system remains responsive and efficient.

Q2. Differentiate between preemptive and non-preemptive scheduling by examining the specific process state transitions that trigger a scheduling decision. Why is non-preemptive scheduling considered "cooperative"?

Answer: CPU scheduling decisions occur at four specific transition points in a process's life:

1. When a process switches from **running to waiting** (e.g., an I/O request).
2. When a process switches from **running to ready** (e.g., an interrupt).
3. When a process switches from **waiting to ready** (e.g., I/O completion).
4. When a process **terminates.**

Non-preemptive scheduling occurs only under conditions 1 and 4. In this mode, once the CPU is allocated to a process, that process keeps the CPU until it voluntarily releases it by terminating or switching to a waiting state. It is "cooperative" because the scheduler waits for the process to yield.

Preemptive scheduling, on the other hand, can occur during conditions 2 and 3. Here, the OS can forcibly remove a running process from the CPU to allocate it to a higher-priority task. This approach is standard in modern operating systems (like Windows 95+ and Macintosh OS) to ensure responsiveness, whereas older systems like Windows 3.x relied on non-preemptive methods

Q3. The "Dispatcher" is a distinct component from the scheduler. Elucidate the specific responsibilities of the dispatcher and explain how "dispatch latency" impacts system performance.

Answer: It is important to distinguish between selection and execution. The scheduler selects which process should run next, but the dispatcher is the module that actually gives control of the CPU to that selected process.

The dispatcher performs the low-level mechanics of the hand-over, which involves:

1. **Switching context**: Saving the state of the old process and loading the state of the new one.
2. **Switching to user mode**: Transitioning from kernel mode back to user mode so the application can run.
3. **Jumping:** Moving the instruction pointer to the proper location in the user program to resume execution.

This process is not instantaneous. The time taken for the dispatcher to stop one process and start another is called **dispatch latency**. This latency is pure overhead—no useful work is done during this time. Therefore, minimizing dispatch latency is crucial; if it is too high, the system spends more time shuffling processes than actually executing them.

Q4. The First-Come, First-Served (FCFS) algorithm is technically simple but can lead to the "Convoy Effect." Analyze this phenomenon and explain why FCFS is generally considered unsuitable for time-sharing systems.

Answer: FCFS is the simplest scheduling algorithm: processes are assigned the CPU in the exact order they request it. It is **non-preemptive**, meaning once a process gains the CPU, it retains control until it voluntarily terminates or waits for I/O.

The **Convoy Effect** occurs when a single CPU-bound process (a process with a very long CPU burst) effectively "clogs" the system. Several shorter I/O-bound processes may get stuck waiting behind this single long process. Instead of quickly executing their short bursts and returning to I/O, they idle in the ready queue, leading to poor resource utilization and high average waiting times.

This characteristic makes FCFS problematic for **time-sharing systems**. In such environments, it is critical that every user or process receives a share of the CPU at regular intervals to maintain responsiveness. FCFS cannot guarantee this, as one process can monopolize the CPU for an extended period, making the system appear unresponsive to other users.

Q5. Consider a set of processes *P*1, *P*2, *P*3 arriving at the same time (t=0) with burst times of 24ms, 3ms, and 3ms respectively. Compare the Average Waiting Time (AWT) if they are scheduled using FCFS (order *P*1, *P*2, *P*3) versus FCFS (order *P*1, *P*2, *P*3). What does this comparison reveal about the sensitivity of FCFS?

Answer:

This scenario highlights the extreme sensitivity of FCFS to arrival order.

- Case 1 (*P*1, *P*2, *P*3): *P*1 (24ms) runs first. *P*2 waits 24ms. *P*3 waits 27ms (24+3).
  - AWT = (0 + 24 + 27) / 3 = 17ms.
- Case 2 (*P*1, *P*2, *P*3): *P*2 (3ms) runs first. *P*3 waits 3ms. *P*1 waits 6ms (3+3).
  - AWT = (0 + 3 + 6) / 3 = 3ms .

The comparison reveals that the AWT under FCFS is not minimal and can vary substantially depending on whether CPU-bound processes arrive before or after I/O-bound processes. A simple reordering of the same tasks reduced the waiting time by over 80%, illustrating why FCFS is risky

for systems with variable burst lengths.

Lecture 8 - CPU Scheduling Algorithms 2

Q1. Priority Scheduling assigns the CPU based on rank rather than arrival time. Explain the convention used to define "highest priority" in most systems and discuss the critical problem of "Starvation" inherent to this algorithm. How does "Aging" resolve this issue?

Answer: In Priority Scheduling, a priority number (integer) is associated with each process. The convention typically followed is that a smaller integer denotes a higher priority; thus, the CPU is allocated to the process with the smallest integer value.

The inherent weakness of this algorithm is indefinite blocking, also known as starvation. In a heavily loaded system, a steady stream of higher-priority processes can prevent a low-priority process from ever gaining access to the CPU. A low-priority task could literally wait indefinitely without ever executing.

To solve this, schedulers implement a technique called Aging. This involves gradually increasing the priority of a process as it sits in the wait queue. For example, if a process's priority number is decremented (improved) every few minutes, it will eventually become the highest-priority process in the system and be executed, regardless of new arrivals.

Q2. Conceptually, Shortest-Job-First (SJF) can be categorized as a specific type of Priority Scheduling. Analyze the relationship between these two algorithms. What determines the "priority" in an SJF context?

Answer: SJF is essentially a specific application of the general Priority Scheduling mechanism. In standard priority scheduling, the priority is an arbitrary number assigned based on external criteria (like user importance or payment).

In the case of SJF, the priority is internally derived from the process's characteristics. Specifically, the priority is the inverse of the predicted next CPU burst time. The shorter the predicted burst, the higher the priority (or the lower the priority value) the process is assigned. Therefore, SJF creates an ordering where the "most important" jobs are simply the ones that can finish the quickest.

Q3. The performance of the Round Robin algorithm is highly sensitive to the size of the Time Quantum. Contrast the system behavior when the quantum is "extremely large" versus when it is "extremely small." What is the impact on context switching?

Answer: The choice of time quantum dictates the nature of the scheduler:

● Extremely Large Quantum: If the quantum is large enough to cover the longest CPU burst in the system, the RR policy degenerates into a standard FCFS policy. The overhead of round-robin management remains, but the benefit of responsiveness is lost.

● Extremely Small Quantum: If the quantum is too small, the system suffers from excessive overhead. For example, if the quantum is 1 time unit and a process needs 10 units, the system must perform 9 context switches. Since a context switch takes time (typically < 10 microseconds), performing them too frequently wastes significant CPU time on administrative tasks rather than executing user code.

Thus, the quantum must be large enough relative to the context-switch time to be efficient, but small enough to maintain interactivity.

Lecture 10 - CPU Scheduling Algorithms 3

Q1. How does the Multilevel Feedback Queue (MLFQ) scheduler improve upon the standard Multilevel Queue approach? specifically, detail the five parameters that define an MLFQ scheduler and explain how "aging" is implemented in this system.

Answer: The standard Multilevel Queue algorithm suffers from inflexibility because processes are permanently assigned to a queue. The Multilevel Feedback Queue (MLFQ) improves upon this by allowing processes to move between queues based on their behavior.

An MLFQ scheduler is defined by five key parameters:

1. The number of queues.
2. 2. The scheduling algorithm used for each queue.
3. 3. The method to determine when to upgrade a process to a higher-priority queue.
4. 4. The method to determine when to demote a process to a lower-priority queue.
5. 5. The method to determine which queue a process enters when it first needs service.

This flexibility allows the system to implement aging to prevent starvation. If a low-priority process waits too long, it can be moved to a higher-priority queue. Conversely, processes that consume too much CPU time (CPU-bound) can be demoted to lower-priority queues, leaving the high-priority queues for interactive (I/O-bound) processes.

Q2. Consider a Multilevel Feedback Queue system with three queues: $Q0$ (RR, 8ms quantum), $Q1$ (RR, 16ms quantum), and $Q2$ (FCFS). Trace the path of a long CPU-bound process entering this system and explain how preemption is handled between these queues.

Answer: In this specific MLFQ configuration, a new process enters $Q0$ and is served on a FirstCome, First-Served basis. When it gains the CPU, it receives a time quantum of 8 milliseconds .

If the process is CPU-bound and does not finish within this 8ms quantum, it is demoted to the tail of $Q1$. Once in Q_1, it waits until Q_0 is empty. When it eventually runs, it receives a larger time quantum of 16 milliseconds. If it still fails to complete its burst within this 16ms allocation, it is preempted again and demoted to Q_2, where it is processed using a standard FCFS algorithm alongside other background tasks .

Preemption in this system is hierarchical. The scheduler prioritizes higher queues absolutely. If a process arrives for Q_1, it will preempt a running process from Q_2. Furthermore, a process running in Q_1 will itself be preempted if a new process arrives in Q_0, ensuring the system remains responsive to new, short tasks.

Lecture 11 - Introduction to shell and cron

Q1. Analyze the operational flow of a Shell when a user enters a command. specifically, how does the shell distinguish between a "Shell Variable" and an "Environment Variable" in terms of scope and inheritance?

Answer: When a user types a command, the shell acts as an interface between the user and the kernel. It parses the input into tokens (command name, options, arguments) and locates the executable—either in the current directory or by searching through the directories listed in the $PATH variable.

A critical distinction in shell operation is variable management:

● Shell Variables: These are local to the specific shell instance where they are defined. They are not passed down to child processes. They are often used for ephemeral data within a script or session .

● Environment Variables: These are defined for the current shell but are inherited by any child shells or processes spawned from it. This allows configuration data (like user preferences or system paths) to be passed seamlessly to applications .

● Scope Transition: A shell variable can be promoted to an environment variable using the export command (e.g., export MY_VAR="value"). Without export, a child shell (spawned by typing bash) would not see the variable.

Q2. Explain the significance of the PATH environment variable. What specific problem does it solve for the user, and how does the shell utilize it during command execution?

Answer: The PATH environment variable is a colon-separated list of directories that acts as a roadmap for the shell.

Without PATH, a user would have to type the absolute path of every program they wanted to run (e.g., /usr/bin/ls instead of just ls). The PATH variable solves this by automating the search. When a command is entered, the shell iterates through the directories listed in PATH in order. It executes the first matching executable it finds. This mechanism allows users to run system utilities and custom scripts from any location in the file system without knowing their exact installation path.